THE NATIONAL UNIVERSITY OF LESOTHO
Department of Mathematics and Computer Science · Faculty of Science and Technology
CS3520 Computer Organisation and Architecture · AY 2026/2027

LIMO Group Project · Milestone M1 Submission

LIMO Instruction Set Architecture

Team	Data-Academia
Lab group	A, B and C
Release tag	`m1`
Due	14 October 2026
Page limit	6 pages

#	Member	Student no.	GitHub
1	Thabelo Mapepesa	202321273	`@Kirito-36`
2	Rorisang Mokhubu	202322685	`@Rorisang440`
3	Tsenolo Mokati	202405314	`@pauseclose`
4	Lebohang Soai	202322577	`@MissEpsilon`
5	Kopanelo Ntsatsi	201100530	`@Ntsatsik`
6	Letlao Libete	202321251	`@libetehercules-netizen`
7	Realeboha Mphatle	201902287	`@Reah.mphatle`

Repository	https://github.com/Kirito-36/limo-Data-Academia
Live site	https://kirito-36.github.io/limo-Data-Academia/
Assessed	Part I: ISA specification A1–A5 (40) and design-decision log A6 (20); defended individually at D1 (40).

1 Team Charter and Set-up

Team charter	`https://github.com/Kirito-36/limo-Data-Academia/blob/main/src/tests/examples/docs/Data_Academia_Team_Charter.pdf`
Project board	`https://github.com/users/Kirito-36/projects/1/views/1`

2 ISA Specification

2.1 Overview

	Specification	Rationale
Architecture style	32-bit load–store, based on RV32I. Only `bala` and `beha` touch memory. Separate instruction and data memories.	Keeps the RISC-V structure so the textbook datapath and decoder apply. Cost: no memory operands, so every access needs a load or store. Rejected: a memory-to-memory design.
Register bank	16 GPR/SPR registers r0–r15 (4-bit numbers) plus the PC as a visible SPR.	Halves the register file (512 vs 1024 bits) and narrows the forwarding comparators to 4 bits. Cost: only 13 registers left for data.
Addressing modes	Register, immediate, base + offset, PC-relative.	The minimum needed by our 10 instructions; no indexed or auto-increment modes, to keep the address adder simple.
Memory, endianness, alignment	Byte-addressed, little-endian, word-aligned; memory size 4 GB.	Matches RV32I, so Ripes cross-checks work. Word-aligned accesses avoid byte-lane logic.
Instruction formats	R, I (incl. I-shift), S, B. No U or J.	Every instruction fits these four layouts. No instruction needs a 20-bit immediate or a jump-and-link.
Departures from RV32I	(1) 16 registers: top bit of rs1, rs2, rd is reserved (must be 0). (2) No jal, jalr, U-type, lui or auipc. Field positions, opcodes and funct values are unchanged.	Reserved bits keep the RV32I positions, so each LIMO instruction is a valid RV32I instruction using only x0–x15. Cost: three unused bits per R-type instruction.

2.2 Register File

Number	Sesotho name	Alias	Type	Purpose and convention
r0	noto	`zero / noto`	SPR	Hard-wired to 0; writes are ignored
r1	lehae	`lehae`	SPR	Return address (convention; nothing writes it automatically in v1.0)
r2	lesupa	`lesupa`	SPR	Stack pointer
r3–r8	nakoana	`n1–n6`	GPR	Temporaries (not preserved)
r9–r12	kenyo	`k1–k4`	GPR	Arguments and results
r13–r15	poloko	`p1–p3`	GPR	Saved registers (preserved)
PC	sesupo	–	SPR	Address of the current instruction (not in the register file; only branches change it)

Name rationale: lehae = home, where a call returns to; lesupa = the one that points; nakoana = short-lived; kenyo = what is put in; poloko = safekeeping.

2.3 Instruction Formats

bits	31..25	24	23..20	19	18..15	14..12	11	10..7	6..0
R	funct7	0	rs2	0	rs1	funct3	0	rd	opcode
I	imm[11:5]	imm[4]	imm[3:0]	0	rs1	funct3	0	rd	opcode
I-shift	funct7	shamt[4]	shamt[3:0]	0	rs1	funct3	0	rd	opcode
S	imm[11:5]	0	rs2	0	rs1	funct3	imm[4]	imm[3:0]	opcode
B	imm[12|10:5]	0	rs2	0	rs1	funct3	imm[11]	imm[4:1]	opcode

A **0** is a reserved bit that must be 0. **Freed bits:** because registers need 4 bits, the top bit of each register field (bits 11, 19, 24 in R) is reserved rather than reused. In S and B, bit 11 carries immediate bits, so only bits 19 and 24 are reserved. **Immediate generator:** I = sext(inst[31:20]); S = sext(inst[31:25], inst[11:7]); B = sext(inst[31], inst[7], inst[30:25], inst[11:8], 0).

Immediate ranges: I: −2048 to 2047; S: −2048 to 2047; shamt: 0 to 31; B (branch reach in bytes): −4096 to +4092, multiples of 4, counted from the branch's own address (1024 instructions).

2.4 Instruction Set and Sesotho Mnemonics

Class	Mnemonic	Meaning	Syntax	Operation	Fmt	RV32I
Arithmetic	**`kopanya`**	add (join together)	`kopanya rd, rs1, rs2`	R[rd] ← R[rs1] + R[rs2]	R	`add`
Arithmetic	**`tlosa`**	subtract (remove)	`tlosa rd, rs1, rs2`	R[rd] ← R[rs1] − R[rs2]	R	`sub`
Arithmetic	**`eketsa`**	add immediate (add more)	`eketsa rd, rs1, imm`	R[rd] ← R[rs1] + sext(imm)	I	`addi`
Logic	**`kapa`**	bitwise OR	`kapa rd, rs1, rs2`	R[rd] ← R[rs1] | R[rs2]	R	`or`
Logic	**`le`**	bitwise AND	`le rd, rs1, rs2`	R[rd] ← R[rs1] & R[rs2]	R	`and`
Logic	**`suthisa`**	shift left (cause to move)	`suthisa rd, rs1, shamt`	R[rd] ← R[rs1] << shamt	I	`slli`
Memory	**`bala`**	load word (read)	`bala rd, imm(rs1)`	R[rd] ← M[R[rs1] + sext(imm)]	I	`lw`
Memory	**`beha`**	store word (put)	`beha rs2, imm(rs1)`	M[R[rs1] + sext(imm)] ← R[rs2]	S	`sw`
Branch	**`lekanya`**	branch if equal (compare)	`lekanya rs1, rs2, label`	if R[rs1] = R[rs2]: PC ← PC + offset	B	`beq`
Branch	**`phapanyo`**	branch if not equal (difference)	`phapanyo rs1, rs2, label`	if R[rs1] ≠ R[rs2]: PC ← PC + offset	B	`bne`

Lexical rules. Mnemonics and register names are case-insensitive and plain ASCII letters. Comments start with #; a label is name: at the start of a line. Registers are accepted as a number (r3), an alias (n1, k1, p1, noto) or a Sesotho name.

Immediates are decimal or hex (0x…); memory operands are offset(base). Branches take a label; the assembler computes the byte offset and reports an error if it is not a multiple of 4 or is outside −4096..+4092. **Apostrophes:** none of our mnemonics or registers contain one, so instructions are plain ASCII. The straight (U+0027) and curly (U+2018, U+2019) apostrophes are treated as the same character and normalised to U+0027, so text pasted from a phone still works. Labels may contain apostrophes and they are significant (a label with ts' differs from one with ts), because silently deleting a letter would change the Sesotho word. There are no character literals, so there is no clash.

2.5 Encoding Worked Examples

Instruction	Fmt	Field values	Binary (bit 31 … 0)	Hex
`kopanya r3, r1, r2`	R	funct7=0, rs2=2, rs1=1, funct3=0, rd=3, op=0110011	`0000000 00010 00001 000 00011 0110011`	`0x002081B3`
`eketsa r3, r1, 5`	I	imm=5, rs1=1, funct3=0, rd=3, op=0010011	`000000000101 00001 000 00011 0010011`	`0x00508193`
`beha r3, 8(r4)`	S	imm=8, rs2=3, rs1=4, funct3=2, op=0100011	`0000000 00011 00100 010 01000 0100011`	`0x00322423`
`lekanya r3, r4, +8`	B	imm=+8, rs2=4, rs1=3, funct3=0, op=1100011	`0 000000 00100 00011 000 0100 0 1100011`	`0x00418463`

Extra checks: suthisa r3, r1, 2 = 0x00209193; bala r3, 4(r4) = 0x00422183; lekanya r3, r4, −8 = 0xFE418CE3. Binary strings show each register as 5 bits including the reserved top bit; for B the groups are imm[12], imm[10:5], rs2, rs1, funct3, imm[4:1], imm[11], opcode.

Opcode and funct values (same as RV32I)

Instruction	Format	opcode	funct3	funct7
`kopanya`	R	`0110011`	`000`	`0000000`
`tlosa`	R	`0110011`	`000`	`0100000`
`kapa`	R	`0110011`	`110`	`0000000`
`le`	R	`0110011`	`111`	`0000000`
`eketsa`	I	`0010011`	`000`	–
`suthisa`	I (shift)	`0010011`	`001`	`0000000`
`bala`	I	`0000011`	`010`	–
`beha`	S	`0100011`	`010`	–
`lekanya`	B	`1100011`	`000`	–
`phapanyo`	B	`1100011`	`001`	–

Load/store addresses must be multiples of 4. Misaligned addresses and non-zero reserved bits are undefined (LIMO has no exceptions). There is no multiply, divide, floating point or CSR support.

3 Design-Decision Log

(a) Register-file size. With 16 registers a number needs 4 bits, so rd, rs1 and rs2 each have one reserved bit (three in an R-type) while fields stay where RV32I puts them. We gain a 512-bit register file instead of 1024, 16:1 read multiplexers and 4-bit forwarding comparators instead of 5-bit. We lose registers: r0, r1 and r2 are taken, leaving 13 for data. Our three programs use at most seven (r3 to r9), so we do not feel the limit yet, but nested loops with saved values would hit it quickly.

(b) Architectural or microarchitectural? Architectural registers are the programmer-visible ones: the PC and r0–r15 (plus the two memories). Microarchitectural ones are hidden: the IR, the pipeline registers IF/ID, ID/EX, EX/MEM and MEM/WB, control signals and the hazard and forwarding units. The ISA must not mention them because it is the contract between software and every implementation. Program 1 ends with r3 = 15 and Mem[0] = 15 on a single-cycle machine (18 cycles) or our pipeline (30); only timing changes. If the ISA fixed the pipeline registers, nobody could change the number of stages without breaking old programs.

(c) Branch resolution. We resolve branches in EX, where the ALU compares rs1 and rs2 and the target is computed. A taken branch discards the two instructions behind it. Program 1 loses 4 × 2 = 8 of its 30 cycles and Program 2 loses 4 of its 33. Resolving in ID would waste only one instruction, but needs its own comparator and extra forwarding paths into ID, and a branch right after its operand producer (eketsa then phapanyo in Program 1)

would still stall a cycle. We picked EX because it needs no extra hardware and reuses the ALU.

(d) Load-use. A load has its data only at the end of MEM, but the next instruction needs it at the start of EX. Without a stall, kopanya would be in EX in cycle 4 while bala is still reading memory, so no forwarding wire can supply the value. We insert one bubble, and kopanya then gets the value from MEM/WB in cycle 5. Program 2 has one such stall per iteration (3 in total). Moving eketsa n2, n2, 4 between the two instructions would remove them.

Clock cycle	1	2	3	4	5	6	7
`bala n5, 0(n2)`	IF	ID	EX	MEM	WB		
`kopanya n4, n4, n5`		IF	ID	ID (stall)	EX	MEM	WB

(e) A flags register? RISC-V has none: a branch compares two registers itself, so no hidden state links an arithmetic instruction to a later branch. Flags would let one tlosa feed several branches, but every arithmetic instruction would write them, creating a new RAW dependency needing its own forwarding and extra state to save. LIMO has only lekanya and phapanyo, each comparing in one instruction (Program 1's loop test needs no setup), so flags would add hazards and save nothing.

(f) What breaks first? Our guess, still to be measured: immediate range first, then register count, then branch reach. With no U-type instruction, eketsa reaches only −2048 to 2047, so bigger constants and addresses need eketsa and suthisa plus an extra register. Then 13 usable registers limit nested loops. Branch reach of ±4 KiB (1024 instructions) should last longest, since our longest loop body is four instructions. So far our programs only use values 0 to 30 and hit none of these limits.

4 Sample LIMO Programs

Assumptions for cycle counts: five stages, full forwarding, one stall for a load-use, branches resolved in EX with predict-not-taken (taken = 2 flushed), 4 cycles to fill the pipeline. n1–n6 = r3–r8, k1 = r9. Results were worked out by hand and still need cross-checking in Ripes after translating to RV32I.

Program 1: Sum of 1 to 5. Loop with a taken branch, adds and a store.

```
 1          eketsa   n1, noto, 0       # n1 = sum = 0
 2          eketsa   n2, noto, 5       # n2 = counter = 5
 3  loop:   kopanya  n1, n1, n2        # sum = sum + counter
 4          eketsa   n2, n2, -1        # counter = counter - 1
 5          phapanyo n2, noto, loop    # repeat while counter != 0
 6          beha     n1, 0(noto)       # Mem[0] = sum
```

Expected final state: r3 = 15, r4 = 0, Mem[0] = 15. 18 instructions, branch taken 4 times (8 flushed): 18 + 4 + 8 = 30 cycles.

Program 2: Sum of a three-element array. Store three values, then loop over them with loads (load-use stall).

```
 1          eketsa   n1, noto, 10
 2          beha     n1, 0(noto)       # Mem[0] = 10
 3          eketsa   n1, noto, 20
 4          beha     n1, 4(noto)       # Mem[4] = 20
 5          eketsa   n1, noto, 30
 6          beha     n1, 8(noto)       # Mem[8] = 30
 7          eketsa   n2, noto, 0       # n2 = pointer
 8          eketsa   n3, noto, 12      # n3 = end address
 9          eketsa   n4, noto, 0       # n4 = sum
10  loop:   bala     n5, 0(n2)         # n5 = Mem[pointer]
11          kopanya  n4, n4, n5        # sum += n5 (load-use stall)
12          eketsa   n2, n2, 4         # pointer += 4
13          phapanyo n2, n3, loop      # repeat until pointer = end
14          beha     n4, 12(noto)      # Mem[12] = sum
```

Expected final state: r3 = 30, r4 = 12, r5 = 12, r6 = 60, r7 = 30; Mem[0] = 10, Mem[4] = 20, Mem[8] = 30, Mem[12] = 60. 22 instructions, 2 taken branches (4 flushed), 3 load-use stalls: 22 + 4 + 3 + 4 = 33 cycles.

Program 3: Logic, shift, memory and a taken branch. Uses AND, OR, subtract, shift, load, store and lekanya.

```
 1          eketsa   n1, noto, 12      # n1 = 1100b = 12
 2          eketsa   n2, noto, 10      # n2 = 1010b = 10
 3          le       n3, n1, n2        # n3 = 1000b = 8
 4          kapa     n4, n1, n2        # n4 = 1110b = 14
 5          tlosa    n5, n4, n3        # n5 = 14 - 8 = 6
 6          suthisa  n6, n5, 2         # n6 = 6 << 2 = 24
 7          beha     n6, 0(noto)       # Mem[0] = 24
 8          bala     k1, 0(noto)       # k1 = 24
 9          lekanya  k1, n6, ok        # equal: taken (load-use stall)
10          eketsa   n6, noto, 99      # skipped
11  ok:     beha     k1, 4(noto)       # Mem[4] = 24
```

Expected final state: r3 = 12, r4 = 10, r5 = 8, r6 = 14, r7 = 6, r8 = 24, r9 = 24; Mem[0] = 24, Mem[4] = 24 (eketsa n6, noto, 99 never executes). 10 instructions, 1 load-use stall, 1 taken branch (2 flushed): 10 + 4 + 1 + 2 = 17 cycles.

References

S. Harris and D. Harris, *Digital Design and Computer Architecture*.

AI-Use Declaration

Tool	Used for	How we checked or changed the output
Claude (Anthropic)	Drafting the document and simplifying its wording.	Reviewed by the team. See AI_USE.md.

We confirm that every member can explain every part of this submission and the code behind it.

Sprint Note (at most one page; not counted)

Sprint goal: Milestone 1 completed.

Pull requests this sprint: 0.

Member	Role this sprint	Delivered (issue and PR numbers)	PRs reviewed
Thabelo Mapepesa	ISA Architect	–	–
Rorisang Mokhubu	Interface Lead	–	–
Tsenolo Mokati	Pipeline Lead	–	–
Lebohang Soai	Scrum and Release Lead	–	–
Kopanelo Ntsatsi	Assembler Lead	–	–
Letlao Libete	Test Lead	–	–
Realeboha Mphatle	Hazard-Unit Lead	–	–

Note: No issue or pull request numbers are listed because the team was still learning how to navigate GitHub during this sprint.

Roles rotate weekly according to the project requirements.